

1 Introduction

This document outlines how the EAC clears market. We have tested the code by passing in the cleared values of the acceptance ratio of the sell orders and the buy orders which are given by NESO, and determining how closely our calculated MCP is to the actual MCP. It will be going through:

- What NESO has told us through the documentation of its rules
- What we have to interpret using the API data
- How we have cleared the markets according to the rules
- The results of our algorithm

2 NESO Pricing Rules

For each product p and each service window w , a uniform market clearing price (MCP) $\lambda_{p,w}$ is determined.

The feasibility of a market clearing price is governed by surplus constraints. For a sell order o cleared in product p and service window w , the individual surplus s_o is defined as

$$s_o = (\lambda_{p,w} - \pi_o) q_o, \tag{1}$$

where π_o denotes the limit price of order o and q_o its cleared volume.

Individual sell orders such as child orders or substitutable child orders may only be accepted if it has a non negative surplus. A parent order is allowed to be non negative subject to the constraints below.

For any accepted basket b that is not looped, the sum of the surpluses of all accepted orders within the basket must be non-negative. For baskets that are part of a looped family L , individual baskets may have negative surplus, provided that the total surplus across the entire looped family is non-negative.

Buy orders do not impose constraints on market clearing prices and therefore do not directly contribute to the determination of MCPs. Buy orders influence the market outcome only through the welfare maximisation stage, by affecting which sell orders are accepted.

Once surplus feasibility conditions are satisfied, market clearing prices are chosen so as to minimise procurement cost. The procurement cost is defined as

$$\sum_{p,w} \lambda_{p,w} \cdot Q_{p,w}^{\text{sell}}, \quad (2)$$

where $Q_{p,w}^{\text{sell}}$ denotes the total accepted sell volume for product p and service window w for a sell order.

In general, there may exist multiple sets of market clearing prices that all minimise procurement cost while satisfying surplus feasibility. As a result, the pricing problem may admit multiple valid MCP solutions. Additional tie-breaking rules are therefore required to select a unique set of prices consistent with those published by NESO.

Finally, market clearing prices are subject to technical minimum and maximum bounds. These bounds are identical across all products and service windows and are set by NESO. All buy and sell order prices, as well as the resulting market clearing prices (before and after rounding), must lie within these bounds. The market design does not specify a unique rule for resolving price indeterminacy beyond procurement cost minimisation and rounding.

3 Interpretation of NESO API Sell Order Data

There are two key interpretations required when using the NESO API data for the market clearing and pricing. First, the API represents sell orders at a granular level that must be aggregated into baskets, loop families, and multi-product orders. Second, several identifiers serve different semantic roles that must be correctly understood to reconstruct the EAC market structure.

The key fields associated with a sell order in the API are described below.

3.1 Auction and Participant Identifiers

- **auctionID:** Identifies a single auction event. An auction may span multiple service windows. Basket looping is only permitted between baskets that belong to the same `auctionID`. Looping relationships must therefore be interpreted within the scope of a single auction.
- **registeredAuctionParticipant:** Identifies the market participant (competitor) submitting the sell order.
- **auctionUnit:** Identifies the unit owned by the participant that is responsible for delivering the service. All baskets and looped baskets must belong to the same unit.

3.2 Basket and Order Structure

- **basketID:** Identifies the basket to which the sell order belongs. A basket groups together a parent order and its associated child and substitutable child orders. Each basket is defined on a single service window and must contain exactly one parent order.
- **loopedBasketID:** Identifies a loop family rather than a specific basket. Baskets with the same non-null `loopedBasketID` belong to the same looped family. Looping is permitted only between baskets with non-overlapping service windows that belong to the same auction and unit.
- **orderID:** Uniquely identifies a sell order. In practice, a logical sell order may be represented by multiple API rows if it spans multiple products. Such rows must be grouped to form a single multi-product order.
- **orderType:** Specifies whether the order is a `parent`, `child`, or `substitutable_child` order. Parent orders are non-curtable, while child and substitutable child orders are curtable.
- **orderEntryTime:** Timestamp indicating when the order was submitted. This field is useful for grouping fragmented submissions into a single logical multi-product order and for resolving ambiguities in order reconstruction.

3.3 Service and Product Information

- **serviceType**: High-level classification of the service being offered, such as Response or Reserve. This field constrains the set of allowable products for the order.
- **deliveryStart** and **deliveryEnd**: Define the service window for the sell order. These timestamps determine basket mutual exclusivity: baskets with overlapping service windows are mutually exclusive and cannot be accepted simultaneously.
- **auctionProduct**: Identifies the specific ancillary service product being offered (e.g. DMH, DML, NQR, PQR). Each product-service window pair has a distinct market clearing price.
- **productID**: Identifier of each product offered within an auction ID

3.4 Economic Parameters and Clearing Results

- **quantity**: The offered volume of the product in MW. Quantities are required to be integers and are validated against the registered capacity of the unit.
- **priceLimit**: The limit price in £/MW/h at which the participant is willing to provide the service. Prices may be negative and are subject to global minimum and maximum bounds set by NESO.
- **acceptanceRatio**: The proportion of the offered quantity that is accepted by the allocation stage of the auction. The cleared volume is given by the product of the acceptance ratio and the offered quantity. This value is treated as fixed input to the pricing problem.

Correct interpretation and aggregation of these fields is essential to reconstruct the basket structure, loop families, and accepted multi-product orders required for the pricing linear programme. Errors in this interpretation can lead to incorrect surplus constraints, invalid pricing solutions, and deviations from NESO-published results.

3.5 Multi-Product Sell Orders

A multi-product sell order represents a single economic sell order that offers capacity across multiple products at a common price and acceptance ratio. While the EAC market design allows a sell

order to specify multiple product–quantity pairs, NESO does not explicitly define a *multi-product sell order* as an object in the API. Instead, such orders are implicitly represented as multiple API rows, each corresponding to a different product.

This distinction is critically important for pricing. Surplus feasibility constraints are applied at the sell order level rather than independently at the product level. In particular, if two product offers belong to the same economic sell order, a loss incurred on one product may be offset by surplus earned on another. Treating these product offers as separate sell orders would incorrectly require each to have non-negative surplus, thereby over-constraining the pricing problem and leading to distorted market clearing prices.

Since NESO does not explicitly specify how to identify a multi-product sell order, it is necessary to reconstruct these orders from the available sell order data. In this work, we interpret a multi-product sell order as a collection of sell order fragments that share the same economic characteristics and therefore represent a single pricing entity.

Specifically, sell order fragments are grouped into a multi-product sell order if they share the following attributes:

- the same **auctionid**,
- the same **order type** (parent, child, or substitutable child),
- the same **basket identifier**,
- the same **price limit**,
- the same **acceptance ratio**,
- the same **service window** (delivery start and end times),
- the same **order entry time**,
- the same **productID**

while differing only in the **product** and corresponding offered quantity.

This reconstruction allows surplus to be correctly evaluated across all products offered by a single sell order, ensuring that the pricing linear programme accurately reflects the surplus transfer mechanisms implied by the EAC market design.

4 Results

Figure 1 shows the accuracy with which the pricing LP reproduces the NESO-published market clearing prices (MCPs) across the analysed auctions. Accuracy is measured as the proportion of product–service window MCPs that match the published values within a tolerance of £0.01. The LP achieves high accuracy overall, with most auctions exhibiting matching rates at or above 90%. The worst observed cases fall just below 80%, indicating that deviations are limited and isolated rather than systematic.

In addition to reproducing MCPs with high accuracy, the pricing LP consistently yields a weakly lower total procurement cost than that implied by the NESO-published prices, as shown in Figure 2. This behaviour does not indicate that the NESO prices are infeasible or suboptimal. Rather, it reflects the fact that the EAC pricing problem generally admits multiple feasible vectors of market clearing prices that satisfy all surplus feasibility constraints.

While the EAC market design specifies procurement cost minimisation as a criterion for selecting among feasible prices, it does not uniquely determine the MCP when prices are indeterminate. In such cases, NESO publishes a feasible MCP that satisfies all market rules, whereas the pricing LP explicitly selects the procurement-cost- minimising element of the feasible set. Consequently, the observed differences in procurement cost arise from alternative tie-breaking choices within the same feasible region, rather than from a discrepancy in the underlying pricing logic.

Figure 3 further illustrates the distribution of procurement cost reductions obtained by the pricing LP. The reductions are typically small and concentrated close to zero, demonstrating that price indeterminacy has a limited economic impact despite leading to observable MCP differences.

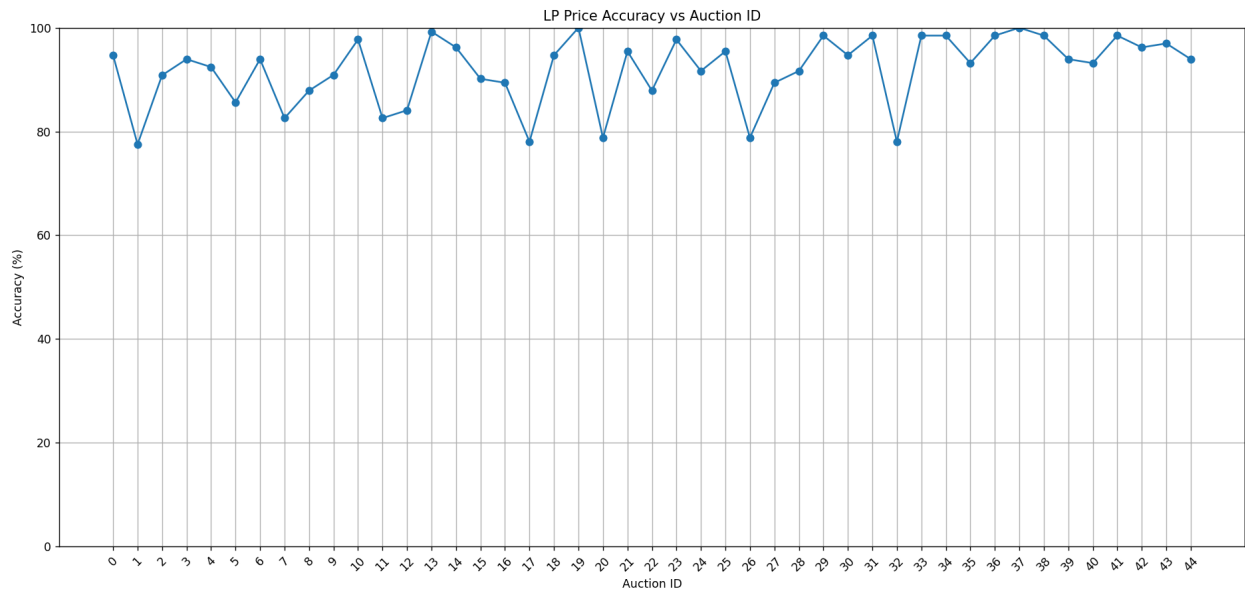


Figure 1: LP price accuracy across auctions, measured as the percentage of MCPs that match the NESO-published values within a tolerance of £0.01.



Figure 2: Relative procurement cost reduction achieved by the pricing LP compared to NESO-published prices across auctions, defined as $(\text{API cost} - \text{LP cost}) / \text{API cost}$. Positive values indicate lower procurement cost under the LP-derived MCPs.

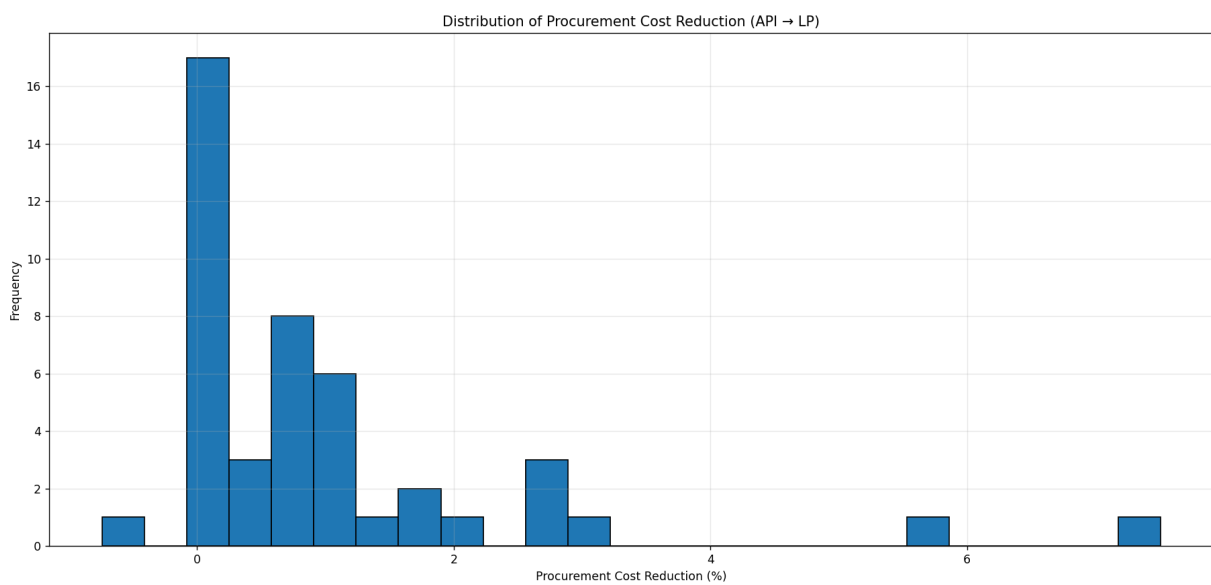


Figure 3: Distribution of procurement cost reductions obtained by the pricing LP relative to NESO-published prices. The concentration of values near zero indicates that, while MCPs may differ due to price indeterminacy, the resulting economic impact is typically small.